LogiTrack: Courier Tracking System

Complete Technical Architecture, Database Specifications & Viva Exam Guide

Database: Oracle AI Database 23c Free (Docker)	**PDB Service:** FREEPDB1	**Schema User:** courier_admin
Backend: Node.js + Express (Port 3000)	**Driver:** node-oracledb (Thin Mode)	**Course Code:** BCSE302L Database Systems

1. System Architecture & 3-Tier Enterprise Design

LogiTrack is built strictly following the standard **3-Tier Enterprise Architecture**. Unlike basic academic projects that manipulate flat files or embedded SQLite databases, this system uses an industrial-grade, client-server relational DBMS.

Tier	Technology	Responsibilities & Interaction Flow
Presentation Tier (Frontend)	HTML5, CSS3, Vanilla JavaScript, Chart.js	Role-based responsive portals (Customer, Admin, Warehouse Staff, Delivery Agent). Makes asynchronous REST calls via apiCall() to port 3000.
Application Tier (Backend)	Node.js, Express, node-oracledb, JWT, bcrypt	Thin coordination layer. Handles password hashing, token validation, and dispatches stored procedure calls to Oracle via TCP port 1521 without raw ad-hoc SQL.
Database Tier (RDBMS)	Oracle Database 23c Free (Docker)	Manages ACID transactions, compiled PL/SQL packages, row-level locks, autonomous sequences, indexes, and automated audit triggers.

2. The '.db File' Question: Why There Is No Local File in the Backend

Key Viva Concept: Students often expect a visible file like database.db inside the backend folder. Understanding why this does not exist in Oracle is vital for explaining RDBMS design to examiners.

- **Embedded Databases (SQLite):** A serverless library where the app directly writes bytes to a single file on disk. It lacks network listeners, concurrent write transactions, stored packages, and multi-user privilege controls.
- **Client-Server RDBMS (Oracle Database):** The Node.js app is strictly a client. It communicates over the network (Oracle Net protocol, port 1521) with an independent database server daemon inside Docker. The server manages data blocks inside binary tablespaces (users01.dbf) at /opt/oracle/oradata/FREE/FREEPDB1/.
- **Real-time State vs. Source Code:** When a customer books a shipment, the database engine commits rows directly into Oracle's memory buffer and tablespace files. It does *not* modify static .sql text files. The .sql files in your repository are DDL/DML deployment blueprints, not runtime storage.

3. End-to-End Role Workflows & Live Lifecycle

A parcel transitions through a strict state machine across 4 distinct user roles:

Stage & Role	Status	Internal PL/SQL & Table Modifications
1. Customer Books Parcel	**BOOKED**	Calls PKG_SHIPMENT.BOOK_SHIPMENT. Atomically inserts records into SHIPMENT, PARCEL, TRACKING, and PAYMENT. Evaluates warehouse capacity using SELECT FOR UPDATE.
2. Warehouse Staff Receives Package	**WAREHOUSE_ RECEIVED**	Appears in Staff's 'Incoming' tab. Staff clicks 'Receive', invoking PKG_WAREHOUSE.RECEIVE_SHIPMENT. Re-checks capacity, acquires lock on WAREHOUSE row, logs AUDIT_LOG.
3. Warehouse Staff Dispatches Hub	**IN_TRANSIT**	Moves to 'Dispatch' tab. Staff clicks 'Dispatch', calling PKG_WAREHOUSE.DISPATCH_SHIPMENT. Trigger TRG_SHIPMENT_STATUS_HISTORY fires automatically.
4. Administrator Assigns Agent	**IN_TRANSIT** (Assigned)	Admin clicks 'Assign Agent' on Shipments table. Invokes PKG_DELIVERY.ASSIGN_DELIVERY. Inserts into DELIVERY table and updates SHIPMENT.Agent_ID.
5. Delivery Agent Doorstep Delivery	**OUT_FOR_ DELIVERY**	Immediately visible on Agent's portal under 'Active Deliveries'. Agent marks 'Out for Delivery' calling PKG_DELIVERY.UPDATE_DELIVERY_STATUS.
6. Delivery Agent Customer Signs	**DELIVERED**	Agent captures receiver name & signature, invoking PKG_DELIVERY.MARK_DELIVERED. Sets Delivery_Date, updates status to DELIVERED, and frees agent vehicle to AVAILABLE.

4. Comprehensive Database Scripts Breakdown (File by File)

All database definitions reside in the database/ directory and compile cleanly into the courier_admin schema:

• 01_create_tables.sql

Defines all 16 tables in dependency order with explicit constraint naming. Implements EER specialization: EMPLOYEE is the supertype, with ADMIN, DELIVERY_AGENT, and WAREHOUSE_STAFF as disjoint subtypes with ON DELETE CASCADE. Enforces 1:1 relationships using UNIQUE foreign keys (e.g. PARCEL.Shipment_ID, PAYMENT.Shipment_ID).

• 02_constraints.sql

Adds cross-table integrity guarantees, complex CHECK constraints (e.g. USER_ACCOUNT role isolation ensuring accounts link to either Customer_ID or Employee_ID), and documents complete database schema integrity.

• 03_sequences.sql

Creates 13 autonomous Oracle sequences (SEQ_CUSTOMER, SEQ_SHIPMENT, SEQ_PARCEL, SEQ_DELIVERY, etc.) ensuring atomic, contention-free surrogate key generation without table locks.

• 04_indexes.sql

B-Tree indexes on frequent lookup and join columns (CUSTOMER.Email, SHIPMENT.Current_Status, SHIPMENT.Customer_ID, TRACKING.Shipment_ID). Includes EXPLAIN PLAN demonstrations showing reduction in cost and table full scans.

• 05_views.sql

8 complex database views for decoupled analytics: VW_CUSTOMER_SHIPMENTS, VW_SHIPMENT_TRACKING, VW_PENDING_DELIVERIES, VW_WAREHOUSE_UTILIZATION (calculating live load percentages), and VW_REVENUE_REPORT (monthly aggregations).

• 06_functions.sql

Pure PL/SQL functions: FN_CALCULATE_SHIPPING_COST (rate formulas based on weight, distance, delivery mode), FN_GET_WAREHOUSE_UTILIZATION, FN_IS_VALID_STATUS_TRANSITION (state machine validator returning 1 or 0), and FN_GET_TOTAL_REVENUE.

• 07_procedures.sql

Standalone demonstration procedures: SP_LOG_AUDIT (autonomous audit recording), SP_UPDATE_LAST_LOGIN, and SP_GENERATE_DELIVERY_REPORT (demonstrating explicit CURSOR declaration, FETCH loop, and record aggregation).

• 08_packages.sql

The core business logic engine organized into 8 modular PL/SQL packages: PKG_AUTH (bcrypt hash comparison, registration transactions), PKG_SHIPMENT (atomic booking with SAVEPOINT and capacity check), PKG_WAREHOUSE (SELECT FOR UPDATE row-locking), PKG_DELIVERY (agent assignment & status progression), PKG_PAYMENT, PKG_CUSTOMER, PKG_ADMIN, and PKG_REPORTS.

• 09_triggers.sql

Database event hooks: TRG_SHIPMENT_STATUS_HISTORY (AFTER UPDATE OF Current_Status recording old and new states), TRG_AUDIT_CUSTOMER_INSERT, TRG_AUDIT_SHIPMENT_INSERT, and TRG_VEHICLE_AVAILABILITY (automatically marks vehicle IN_USE when a delivery is assigned).

• 10_seed_data.sql

Bootstrap seed script creating 4 demo accounts (Customer, Admin, Delivery Agent, Warehouse Staff with bcrypt-hashed passwords for 'Demo@123'), realistic routes, vehicles, warehouses, and initial historical shipments.

• 11_demo_queries.sql

Curated evaluation scripts demonstrating Inner/Left/Right Joins, GROUP BY, HAVING, subqueries, CASE statements, analytical aggregate functions, and Explain Plans.

• 12_transaction_concurrency_demo.sql

Two-session concurrency demonstration verifying Oracle's Row-Level Exclusive Locks (TX), SELECT FOR UPDATE NOWAIT, Two-Phase Locking (2PL), and deadlock detection.

5. How Frontend Buttons Trigger Real SQL (Under the Hood)

To explain to your professor that the UI isn't a mock, trace this exact code path for **Shipment Booking**:

```
1. [Browser: booking.html] Customer selects destination route, package dimensions, delivery type and clicks 'Confirm & Pay'. JavaScript: apiCall('/shipments', { method: 'POST', body: JSON.stringify({...}) }) 2. [Node.js: shipmentController.bookShipment] Express receives payload and executes PL/SQL with bind parameters: BEGIN PKG_SHIPMENT.BOOK_SHIPMENT( :p_customer_id, :p_warehouse_id, :p_route_id, :p_weight, :p_delivery_type, :p_parcel_type, :p_length, :p_width, :p_height, :p_fragile, :p_payment_method, :p_shipment_id, :p_parcel_id, :p_tracking_id, :p_payment_id ); END; 3. [Oracle Database: PKG_SHIPMENT.BOOK_SHIPMENT] a. SAVEPOINT sp_book_shipment; b. SELECT Capacity FROM WAREHOUSE WHERE Warehouse_ID = p_warehouse_id FOR UPDATE; (Locks row) c. v_cost := FN_CALCULATE_SHIPPING_COST(p_weight, v_distance, p_delivery_type); d. INSERT INTO SHIPMENT (...) VALUES (SEQ_SHIPMENT.NEXTVAL, ...); e. INSERT INTO PARCEL (...) VALUES (SEQ_PARCEL.NEXTVAL, ...); f. INSERT INTO TRACKING (...) VALUES (SEQ_TRACKING.NEXTVAL, ...); g. INSERT INTO PAYMENT (...) VALUES (SEQ_PAYMENT.NEXTVAL, ...); h. SP_LOG_AUDIT(...); i. COMMIT; (Persists to Oracle disk datafiles)
```

6. High-Probability DBMS Viva Questions & Model Answers

Study these 12 questions carefully. They represent the exact syllabus concepts examiners test during BCSE302L vivas:

Q1: Why did you use Oracle instead of MySQL or SQLite?

Answer: Oracle is an enterprise-grade client-server RDBMS providing robust support for PL/SQL packages (encapsulating state and procedures), row-level locks via SELECT FOR UPDATE, autonomous audit triggers, sequence generators, and strict ACID transaction compliance. SQLite is an embedded file-based engine that lacks multi-user package architecture and granular concurrency controls.

Q2: How does your system enforce ACID properties during shipment booking?

Answer: Atomicity & Consistency: PKG_SHIPMENT.BOOK_SHIPMENT uses a SAVEPOINT (sp_book_shipment). If any insertion into PARCEL, TRACKING, or PAYMENT fails, the entire transaction issues a ROLLBACK TO SAVEPOINT. Isolation: The origin warehouse row is locked using SELECT FOR UPDATE to prevent lost updates or capacity overselling. Durability: Upon successful completion, an explicit COMMIT writes the transaction to Oracle's online redo log files.

Q3: Where is the EER (Enhanced Entity-Relationship) Specialization in your schema?

Answer: In our EMPLOYEE table. EMPLOYEE serves as the Supertype containing shared attributes (Name, Email, Phone, Salary, Employee_Type). ADMIN, DELIVERY_AGENT, and WAREHOUSE_STAFF are disjoint Subtypes that reference EMPLOYEE(Employee_ID) as both Primary Key and Foreign Key with ON DELETE CASCADE.

Q4: How do you enforce a 1:1 relationship between SHIPMENT and PARCEL or PAYMENT?

Answer: In Oracle, a 1:1 relationship is enforced by placing a UNIQUE constraint on the Foreign Key in the child table. In PARCEL and PAYMENT, Shipment_ID is declared as: CONSTRAINT fk_parcel_shipment REFERENCES SHIPMENT(Shipment_ID) and CONSTRAINT uq_parcel_shipment UNIQUE(Shipment_ID).

Q5: Why did you use PL/SQL Packages instead of writing SQL queries inside Node.js?

Answer: Encapsulation and Separation of Concerns: Moving business logic into packages (PKG_SHIPMENT, PKG_WAREHOUSE) minimizes network roundtrips between Node.js and Oracle. It enables compiled query optimization inside Oracle's Shared Pool, enforces security by preventing SQL injection through bind variables, and guarantees that business integrity rules cannot be bypassed by frontend callers.

Q6: What is the purpose of the TRG_SHIPMENT_STATUS_HISTORY trigger?

Answer: It is an AFTER UPDATE OF Current_Status row-level trigger on SHIPMENT. Whenever a shipment's status changes (e.g. from BOOKED to WAREHOUSE_RECEIVED), it autonomously captures :OLD.Current_Status, :NEW.Current_Status, and the current timestamp, and logs it into SHIPMENT_STATUS_HISTORY for legal tracking auditability.

Q7: What is the default isolation level in Oracle, and what anomalies does it prevent?

Answer: Oracle's default isolation level is READ COMMITTED. It guarantees that a query only sees data committed before the query began. It completely prevents Dirty Reads (reading uncommitted changes from concurrent transactions), while permitting non-repeatable reads and phantom reads unless SERIALIZABLE or explicit row locks are used.

Q8: If I insert a record directly in SQL*Plus, why must I execute COMMIT before it appears on the website?

Answer: Because Oracle uses multi-version concurrency control (MVCC) with Read Committed isolation. Uncommitted modifications exist solely in the local session's undo segment. The Node.js server maintains separate connection pool sessions and cannot see dirty data until an explicit COMMIT writes it to the database.

Q9: How did you implement Concurrency Control and avoid the Lost Update problem?

Answer: In PKG_WAREHOUSE.RECEIVE_SHIPMENT, we execute: SELECT Capacity INTO v_capacity FROM WAREHOUSE WHERE Warehouse_ID = p_id FOR UPDATE. This acquires an exclusive row-level TX lock on the warehouse row. Concurrent staff transactions attempting to receive shipments simultaneously are serialized until the lock is released upon COMMIT.

Q10: What indexes did you create and why?

Answer: We created B-tree indexes on foreign keys and frequently filtered columns (IDX_SHIPMENT_STATUS on Current_Status, IDX_SHIPMENT_CUSTOMER on Customer_ID, IDX_TRACKING_SHIPMENT on Shipment_ID). By default, foreign keys in Oracle are not automatically indexed, which can lead to table-level share locks during parent deletes. Indexing them avoids full table scans.

Q11: How does password security work in your project?

Answer: We do not store plaintext passwords. The application tier uses bcrypt with a salt round of 10 to generate cryptographic one-way hashes. The 60-character hash is stored in USER_ACCOUNT.Password_Hash. During login, bcrypt.compare verifies the entered password against the stored hash.

Q12: What role do Oracle Sequences play in your database?

Answer: Sequences (such as SEQ_SHIPMENT and SEQ_CUSTOMER) provide high-performance, atomic, gap-free surrogate keys. Unlike auto-increment columns in file databases which require locking the table header, Oracle sequences use internal memory caches to generate sequential numbers without transactional bottlenecks.